

Android Camera 360AVM 介绍

文档版本
发布日期

V1.1
2024-07-04

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍 Android 平台 360AVM 模块，内容涵盖 360AVM 方案，环视流程，以及测试方法。

读者对象

本文档主要适用于 Android 平台开发/测试 360AVM 环视功能的相关人员。

缩略语

缩略语	英文全称	中文解释
AVM	Around View Monitor	全景式监控影像系统
MVC	Multi Virtual Camera	多虚拟 Camera
VCH	Virtual Channel	虚拟通道

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.1	2024-07-04	- 新增 2.1.4.1 Android 14 帧重组处理流程。 - 调整文档结构。
V1.0	2024-05-17	第一次正式发布。

关键字

360AVM、帧重组、sensor、YUV、RAW、环视

目 录

1 概述.....	1
2 方案介绍.....	3
2.1 伪单帧.....	3
2.1.1 软件架构.....	3
2.1.2 数据通路.....	5
2.1.3 组帧方式.....	6
2.1.4 重组流程.....	7
2.2 帧同步.....	9
2.3 多虚拟通道.....	10
2.4 多虚拟 Camera 方案.....	11
2.4.1 方案描述.....	11
2.4.2 架构介绍.....	11
2.4.3 配置文件.....	12
3 测试方法.....	16
A Sensor 介绍.....	17
A.1 功能开关.....	17
A.2 点亮配置.....	17
A.3 YUV sensor 和 RAW sensor 差异.....	17
A.3.1 功能差异.....	17
A.3.2 点亮配置差异.....	18
A.3.3 处理流程差异.....	18

图目录

图 1-1 360AVM 框图	1
图 1-2 360AVM 环视效果	2
图 2-1 主要车载转接芯片	3
图 2-2 帧重组优化前软件架构	4
图 2-3 帧重组优化后软件架构	5
图 2-4 帧重组方案	6
图 2-5 组帧方式	6
图 2-6 Android 14 帧重组货架化链路示意图	7
图 2-7 基于 TP2855 sensor Kernel Pipeline 数据处理流程	7
图 2-8 Android 14 驱动货架化基本链路流程图	8
图 2-9 帧重组库接口调用流程	8
图 2-10 Android 10~13 帧重组处理流程	9
图 2-11 MAX 输出格式	10
图 2-12 多虚拟通道处理流程	10
图 2-13 虚拟通道信号	11
图 2-14 多虚拟 Camera 架构图	12
图 2-15 多虚拟 Camera 流程图	12
图 2-16 多虚拟通道配置示例	14
图 2-17 伪单帧模式配置示例	14
图 3-1 cam2test 应用测试界面	16
图 3-2 YUV sensor 开关	17
图 3-3 YUV 和 RAW 格式	18
图 3-4 YUV 和 RAW 处理流程差异	18

表目录

表 2-1 virtual_cam_cfg.json 参数含义	12
---------------------------------------	----

1 概述

Camera 360AVM (Around View Monitor, 全景式监控影像系统) 目前已经被广泛应用在车载中控, 有辅助倒车, 监控行车等功能。

360AVM 框图如图 1-1 所示, 环视效果如图 1-2 所示。

图1-1 360AVM 框图

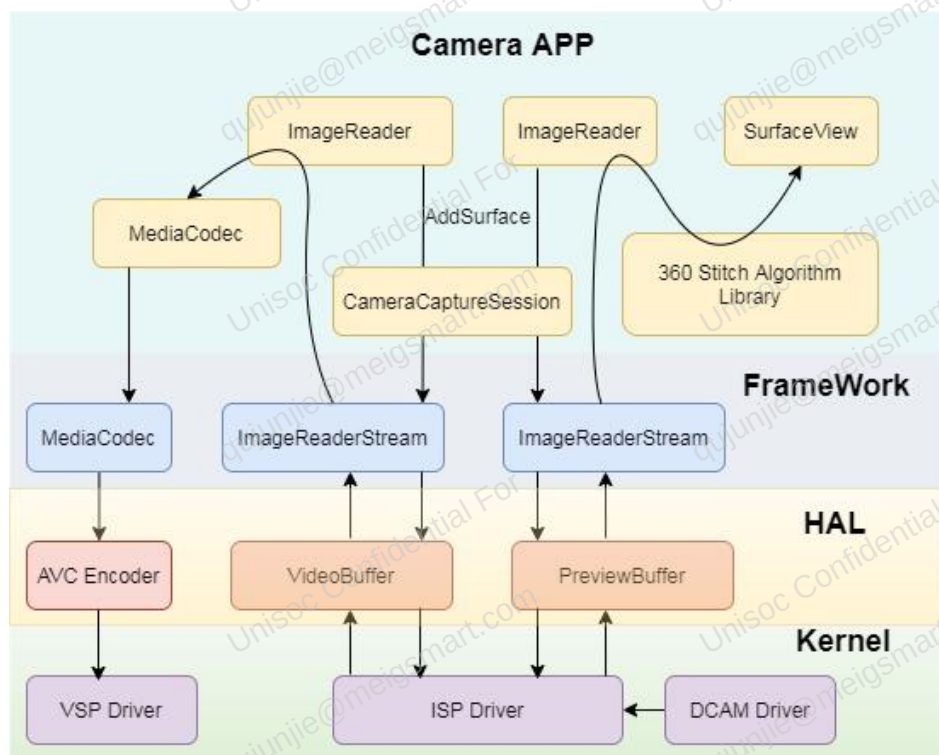


图1-2 360AVM 环视效果



2 方案介绍

Camera 360AVM 是通过多个摄像头捕捉车辆周围的环境的画面并将画面拼接的技术，多个摄像头通过 4 合 1 转接芯片接入平台端。

车载 sensor 分为 RAW sensor 和 YUV sensor，360AVM 环视主要为 YUV sensor，其主要转接芯片如图 2-1 所示。

RAW sensor 和 YUV sensor 相关描述请参见附录 A Sensor 介绍。

图2-1 主要车载转接芯片

Application	Company	ChipSet	Signal	Image Format	Cable	Cost
高端车型	Maxim	MAX9286	GMSL	RAW/YUV	Coax Cable/STP	High
	TI	DS90UB964	FPD-Link III	RAW/YUV	Coax Cable/STP	High
SVS/FC/DMS	Thine	THCV244	V-by-One HS	RAW/YUV	Coax Cable/STP	High
低端车型	TechPoint	TP2854	AHD/CVBS	YUV	Coax Cable/STP	Low
	PixelPlus	PR2100	AHD/CVBS	YUV	Coax Cable/STP	Low
	RICHNEX	RN6864M	AHD/CVBS	YUV	Coax Cable/STP	Low

车载转接芯片主要支持的数据处理模式有：伪单帧、帧同步、多虚拟通道。

2.1 伪单帧

伪单帧是转接芯片将几个 sensor 的数据合并在一起输出，但输出的并不是一个完整正确的图像。

帧重组是将转接芯片转过来的杂乱的帧数据重新组合成对应 Camera 的完整且正确的图像过程，主要针对伪单帧模式。

目前帧重组方案仅在车载平台 Android 10、Android 12、Android 14 上支持。

2.1.1 软件架构

帧重组算法 2022 年 10 月之前在 Kernel 层实现（图 2-2），2022 年 10 月升级到在 HAL 层实现（图 2-3），其主要修改点如下。

- 帧重组迁移到 HAL 层处理，减少 Kernel 资源占用。增加帧数据从 sensor 输出到 DCAM，然后上传到 HAL/OEM 进行帧重组处理，完成后下发到 ISP_dev 模块的数据流转，ISP_dev 完成后续的处理过程。
- HAL 采用 VDSP 进行算法处理，可以优化性能，并降低 CPU 资源的占用。

图2-2 帧重组优化前软件架构

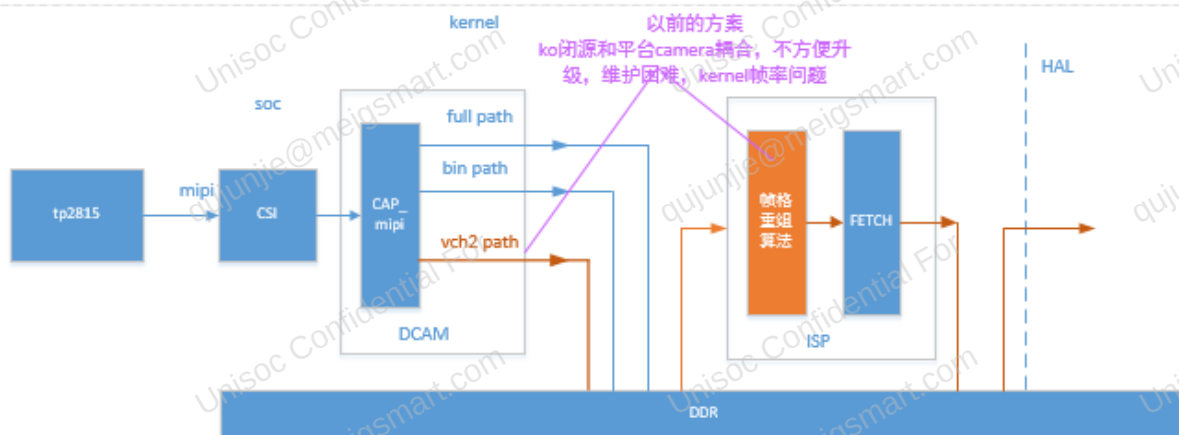
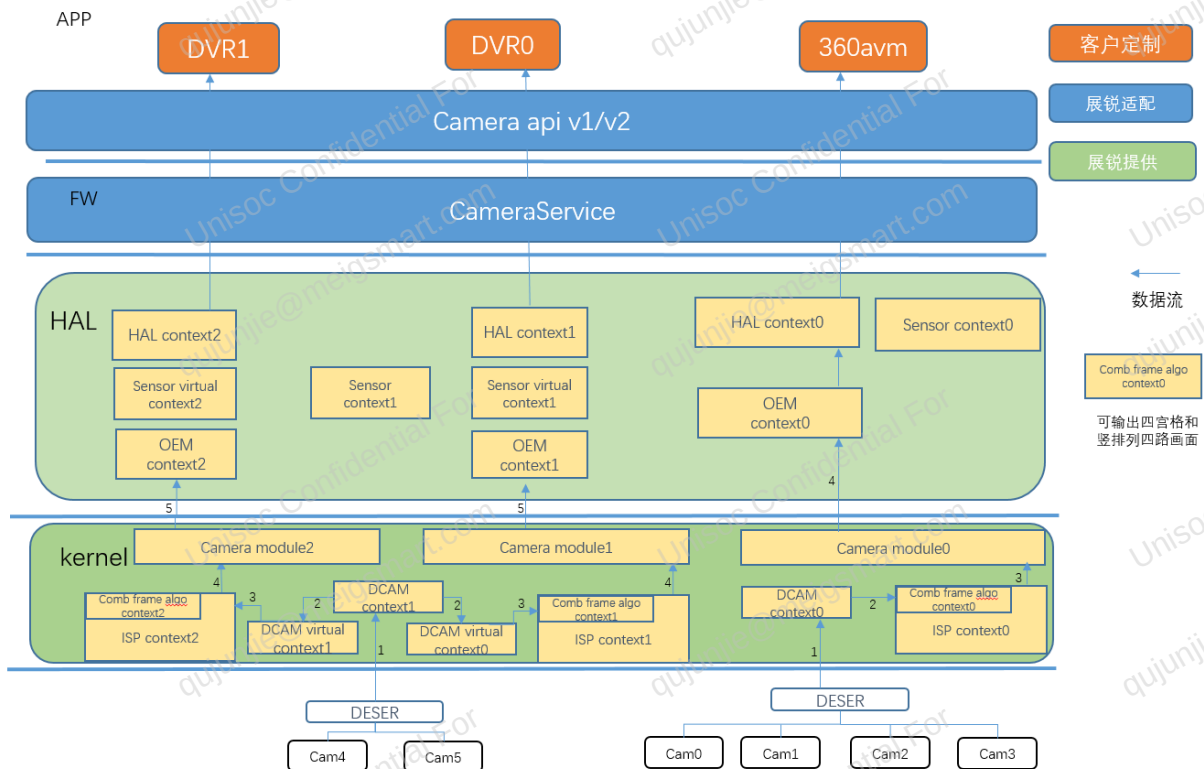
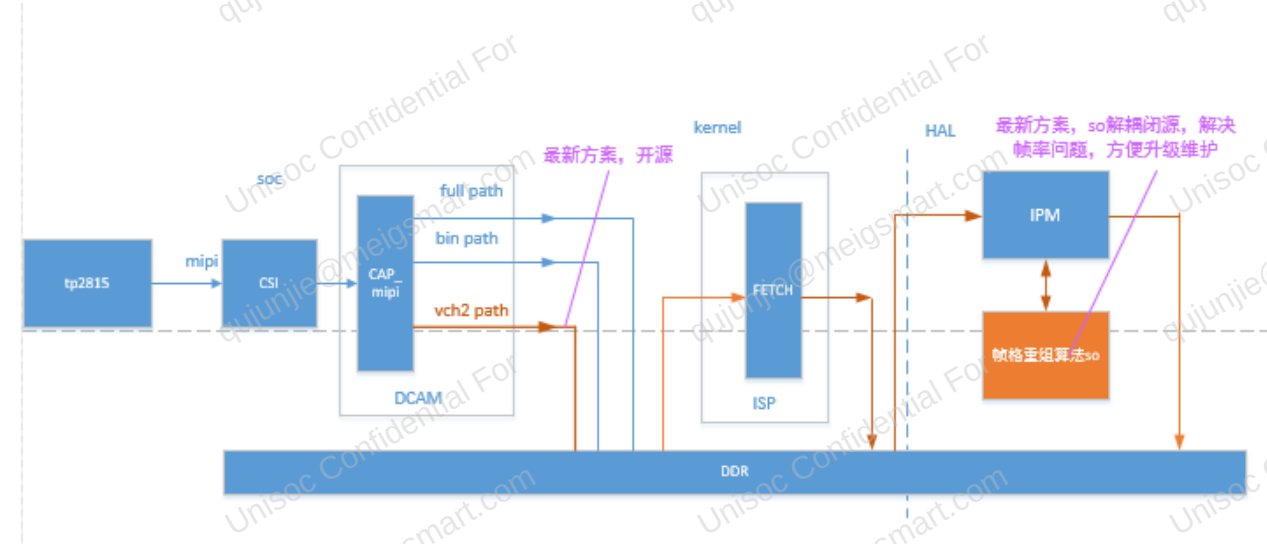
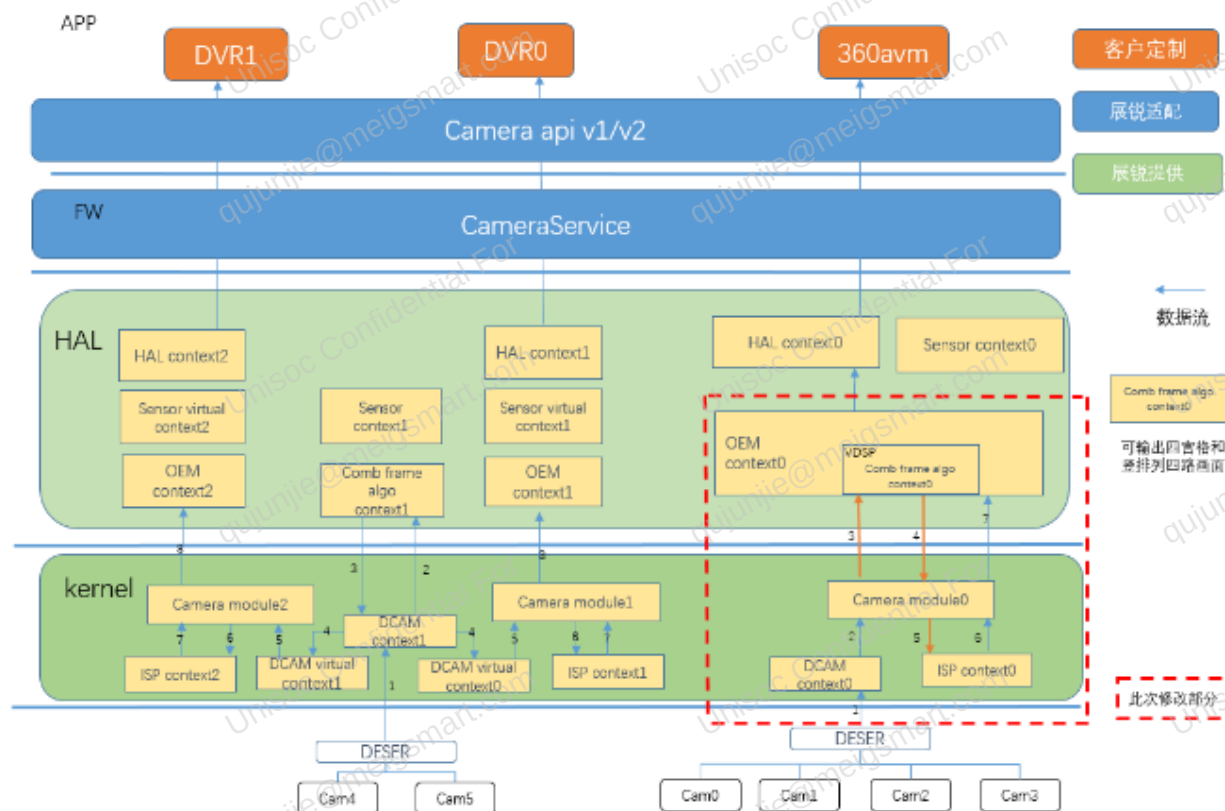


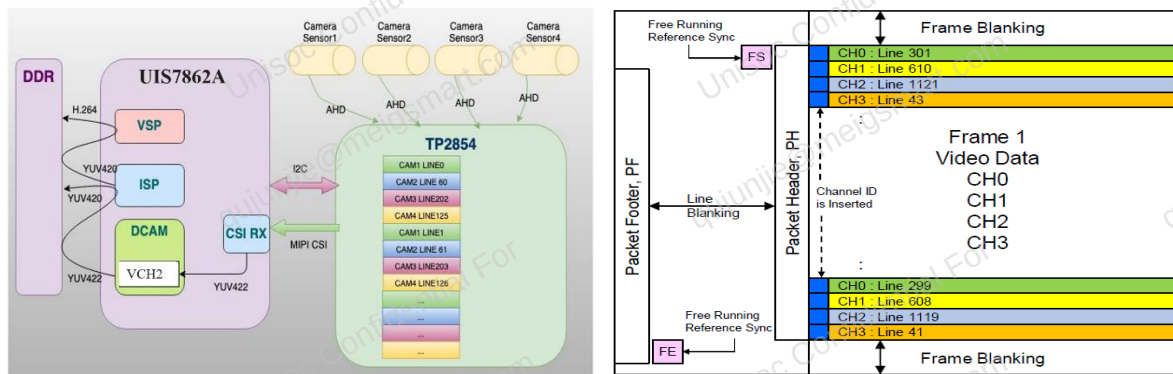
图2-3 帧重组优化后软件架构



2.1.2 数据通路

以 UIS7862A 平台为例，帧重组算法实现如图 2-4 所示。

图2-4 帧重组方案



伪单帧模式 sensor 寄存器配置及使用 VCH2 接收数据的配置在如下文件
vendor/sprd/modules/libcamera/sensor/sensor_drv/classic/转接芯片名/xxxxxxx.h 中。

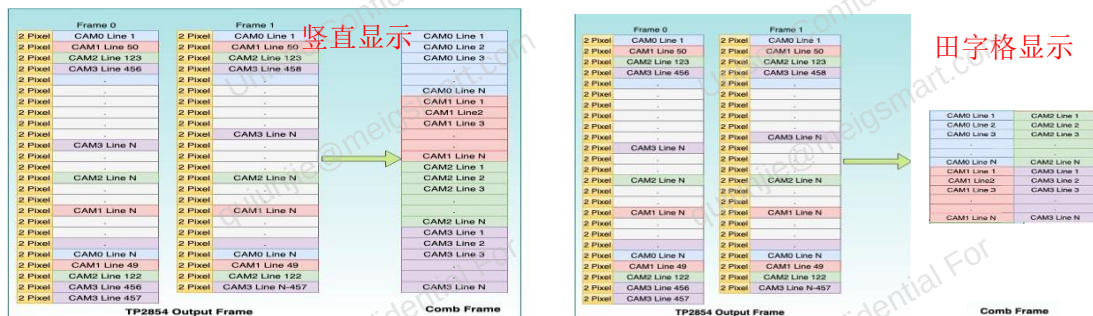
```
vc_id = VC_ID2 //使用VCH2接收数据
```

DCAM VCH2 接收转接芯片输出的 YUV422 图像数据，DCAM VCH2 写到 DDR 的也是 YUV422 图像数据，软件解析每一行的前 2 个 pixel 以获取摄像头 ID 和行信息，然后把相应的行组成对应摄像头的帧数数据。

2.1.3 组帧方式

软件先解析转接芯片输出的 2 帧信息，然后合成 4 个摄像头的一帧图像，组帧方式有两种：竖直方式和田字格方式，具体如图 2-5 所示。

图2-5 组帧方式



组帧方式在文件 vendor/sprd/modules/libcamera/sensor/sensor_drv/classic/转接芯片名/xxxxxxx.h 中定义。

```
out_type = 0, // 0 vertical bar（竖直显示），1 cross bar（田字格显示）
```

- 竖直显示输入、输出信息：
 - 输入数据：YUV422 图像数据， $(1280+2) \times 2880$ ；//2 为 2 个 pixel
 - 输出数据：YUV422 图像数据， $1280 \times 720 \times 4$ ；
- 田字格显示输入、输出信息：
 - 输入数据：YUV422 图像数据， $(1280+2) \times 2880$ ；
 - 输出数据：YUV422 图像数据， $1280 \times 2 \times 720 \times 2$ ；

2.1.4 重组流程

Android 14 Camera 进行了驱动货架化升级，与 Android 13 及之前版本的重组流程有明显的差异。

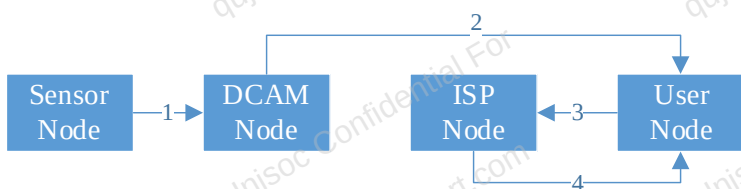
2.1.4.1 Android 14

Android 14 数据链路化的构建要基于流水线、节点、端口和链接这些基本概念将数据通路的需求分解并建立与之对应的拓扑结构。

通常，一条数据通路的需求就是一条流水线，一条流水线会包含若干个虚拟设备节点，虚拟设备节点在运行时会动态绑定物理设备。每一个虚拟设备节点包含若干个端口，每一个端口与物理设备的输入输出通路相对应。每个节点之间的端口可以通过填入链接信息指向其所要链接的节点和端口。通过上述一系列的操作，一条流水线就组装完成。

Android 14 帧重组货架化链路示意图见图 2-6。

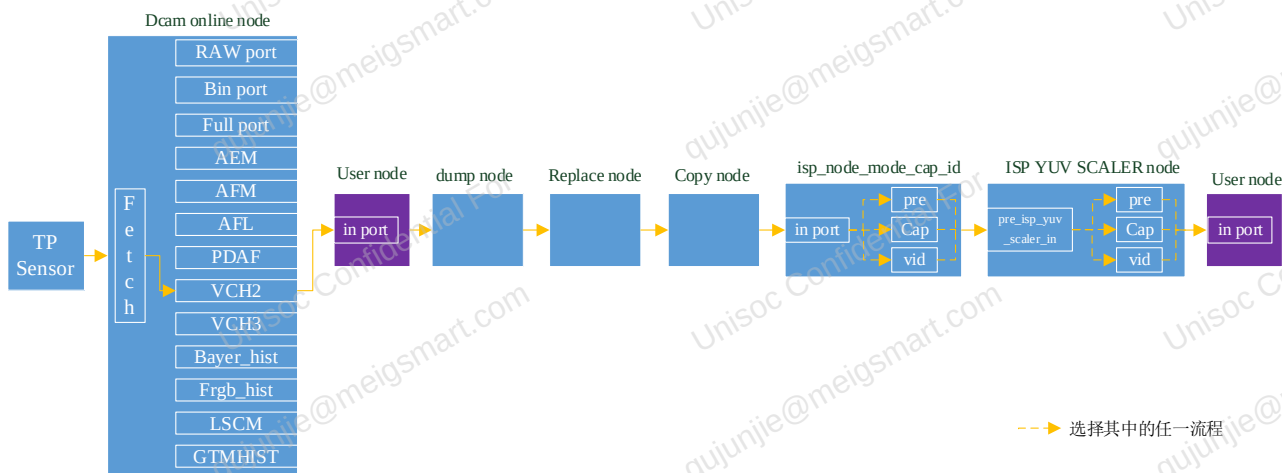
图2-6 Android 14 帧重组货架化链路示意图



Android 14 Camera 驱动货架化后，帧重组数据流为：Sensor > Dcam > HAL/OEM > ISP_DEV > HAL/OEM > FW > App。

基于 TP2855 sensor Kernel Pipeline 数据处理流程见图 2-7。

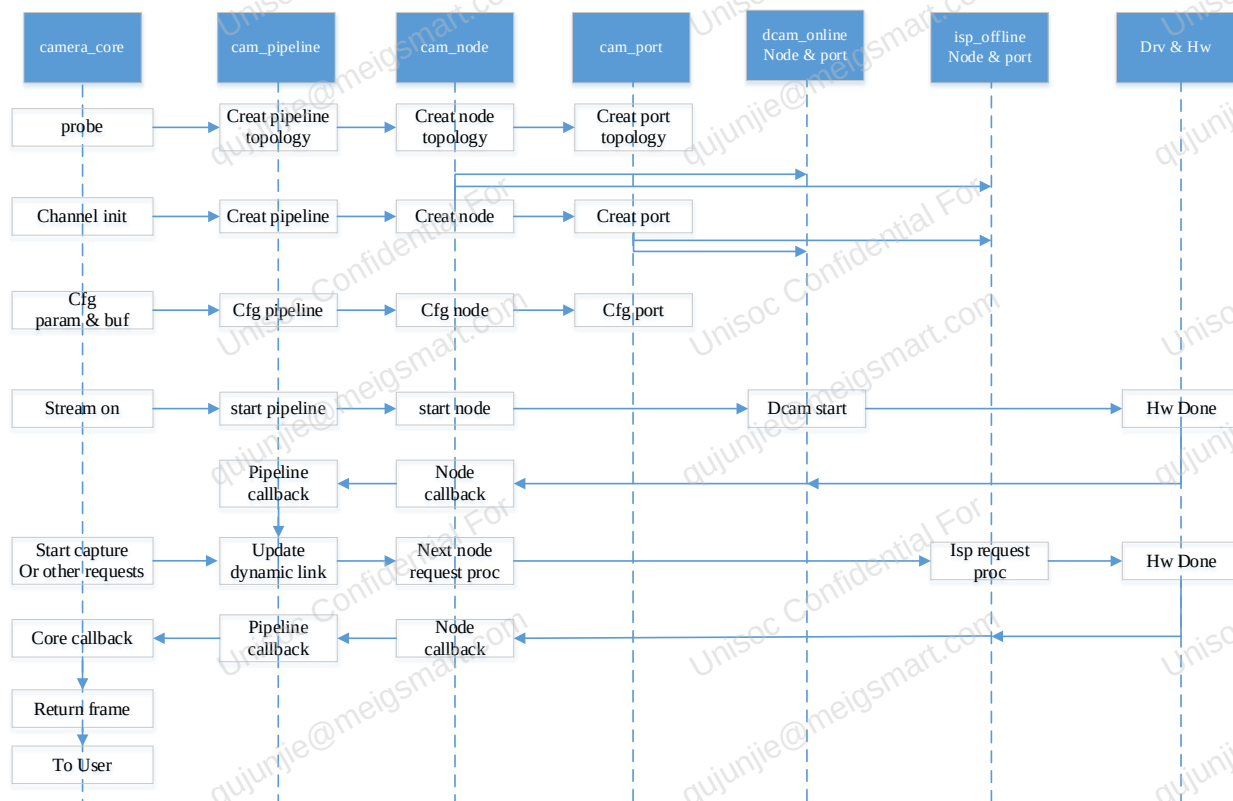
图2-7 基于 TP2855 sensor Kernel Pipeline 数据处理流程



基于 TP2855 sensor 的解析数据直接从 DCAM online VCH2 输出，由 TP2855 将四路 AHD 信号解析成一路 YUV422，并回传给 HAL；HAL 将 YUV422 传送到 pipeline，YUV422 通过算法 recomb 处理后二次传送到 ISP，ISP 将 YUV422 转换成 YUV420 并进行 scaler。

驱动货架化基本链路流程见图 2-8。

图2-8 Android 14 驱动货架化基本链路流程图



当进入帧重组数据处理流程后，会经过驱动货架化基本链路流程。Pipeline_callback 中将数据直接传送到 sprd_image_read，HAL 层获取到底层的原始数据后送入帧重组算法库处理，处理完成后，再将数据返回给 ISP 处理。

帧重组软件算法库接口如下：

- Sprd_comb_frame_open //打开算法库
- Sprd_comb_frame_process //帧重组处理
- Sprd_comb_frame_close //关闭算法库

接口调用流程参见图 2-9。

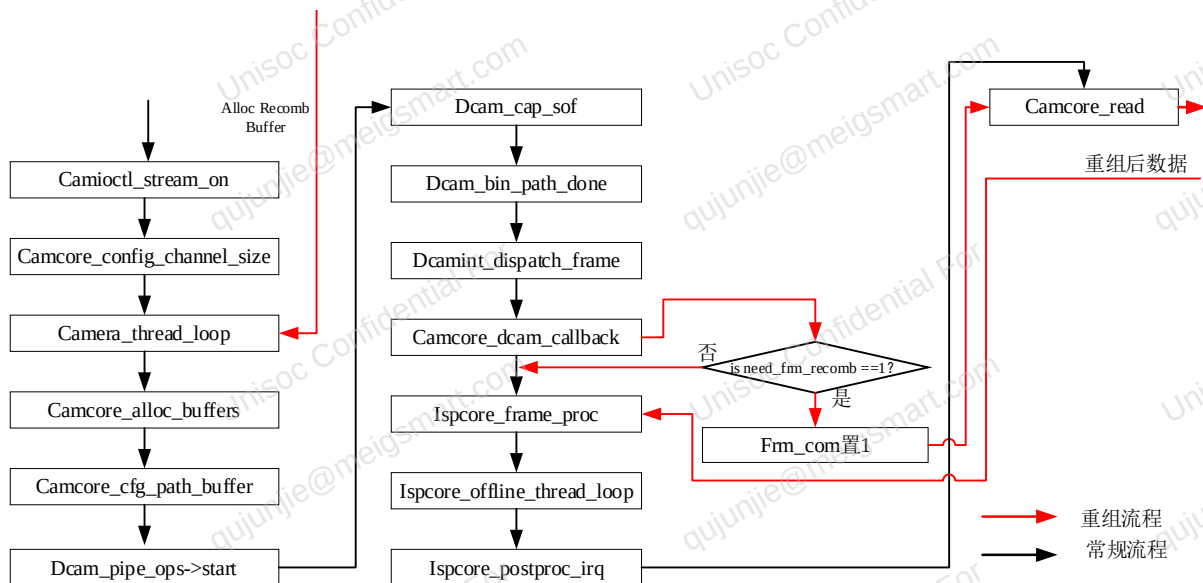
图2-9 帧重组库接口调用流程



2.1.4.2 Android 10~13

Android 10~13 帧重组数据传输及处理流程参见图 2-10。

图2-10 Android 10~13 帧重组处理流程



dcam_callback 中判断需要帧重组时，将数据直接传送到 sprd_image_read，HAL 层获取到底层的原始数据后送入帧重组算法库处理，处理完成后，再将数据返回给 ISP 处理。

Android 10~13 帧重组软件算法库接口及调用流程同 Android 14，具体参见 [2.1.4.1 Android 14](#)。

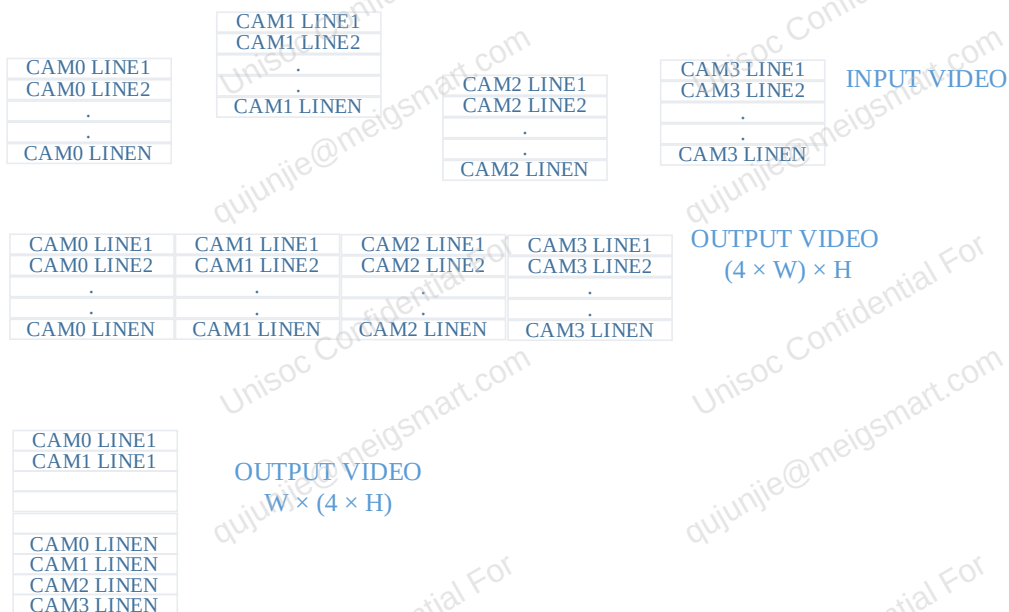
2.2 帧同步

MAX9286 美信解串芯片，带同步功能，可以发送同步信号。

MAX9286 触发模组同步输出 4 帧图像，并把 4 个摄像头的 4 帧图像（每帧图像大小为 $W \times H$ ）组成 1 帧（图像大小为 $(4 \times W) \times H$ 或 $W \times (4 \times H)$ ）后通过 MIPI 发送给平台。输出格式请参见 [图 2-11](#)。

推荐使用 $(4 \times W) \times H$ 方式，无需帧重组，可直接预览。

图2-11 MAX 输出格式

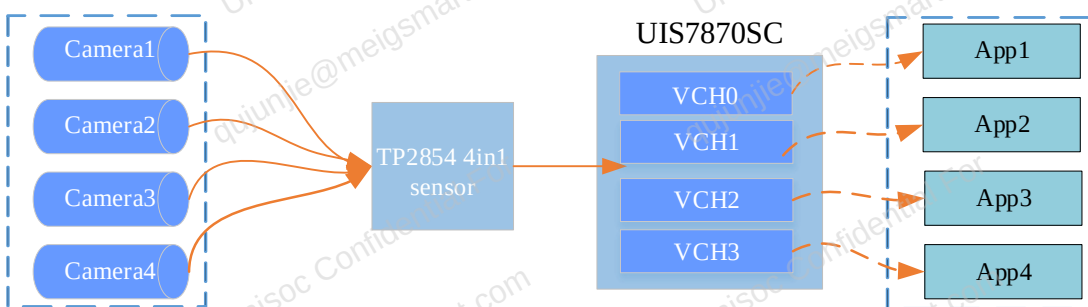


2.3 多虚拟通道

除了帧重组的方式外，还可以使用虚拟通道的方式传输 sensor 数据，虚拟通道的方式只在 UIS7870SC Android 13 上支持，其他平台暂不支持。

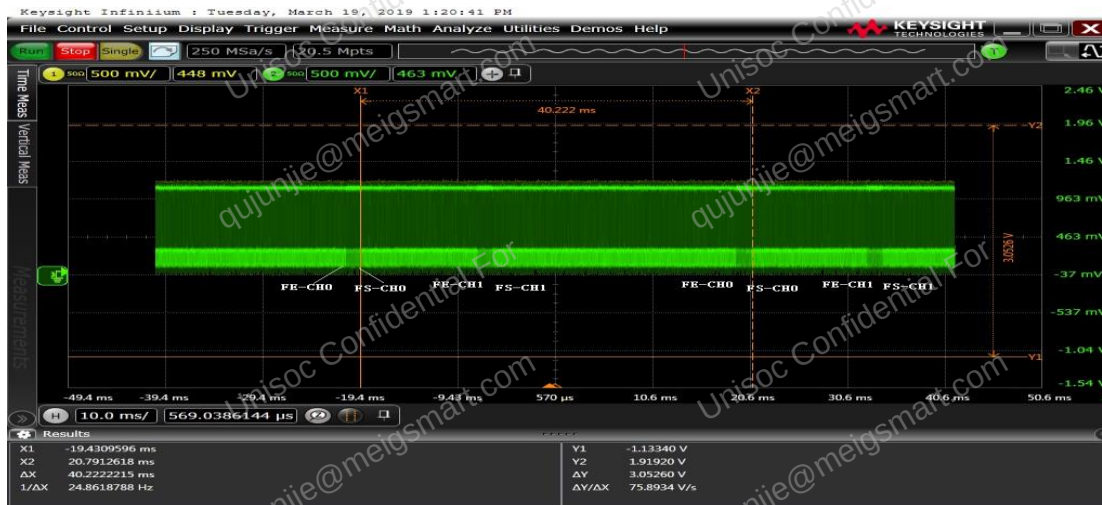
以 TP2854 为例，多虚拟通道处理流程参见图 2-12。

图2-12 多虚拟通道处理流程



硬件测量的虚拟通道 0 和虚拟通道 1 消隐区不重叠波形参见图 2-13。

图2-13 虚拟通道信号



VCH（Virtual Channel，虚拟通道）配置如下：

```
[DCAM_IF_IRQ_INTORAW_PATH TX DONE] = dcamint_rawpath_done  
[DCAM IF_IRQ INTO PDAF PATH TX DONE] = dcamint_pdaf_path_done  
[DCAM IF_IRQINTO_VCH2 PATH TX DONE] = dcamint_vch2_path_done  
[DCAM IF_IRQ INTO VCH3 PATH TX DONE] = dcamint_vch3_path_done
```

如上代码所示，DCAM path 分别使用 rawpath、pdafpath、vch2、vch3。

虚拟通道方式无需进行帧重组，从接收数据区分不同的 sensor id。在向上传递数据时，单路流也不需要数据进行数据拷贝。

2.4 多虚拟 Camera 方案

2.4.1 方案描述

汽车上通常需要同时打开多个 sensor 来监视汽车各个方向的画面，考虑传输距离与信号的稳定性，通常是将多个 sensor 接入转接芯片，转接芯片再接入 ISP 模块。在某种意义上来说转接芯片相当于一个物理 sensor，其出图包含了接入转接芯片的所有内容。在 ISP 处理完图像后，需要将各个物理上对应的 sensor 图像传回各自的 App。

2.4.2 架构介绍

打开多个 Camera id，却只操作一个 sensor，保证各个 App 独立操作互不影响，即一个 App 在预览时，不影响其他 App 打开/关闭相机，并且兼容各自 App 的预览/拍照/录像功能操作。

图2-14 多虚拟 Camera 架构图

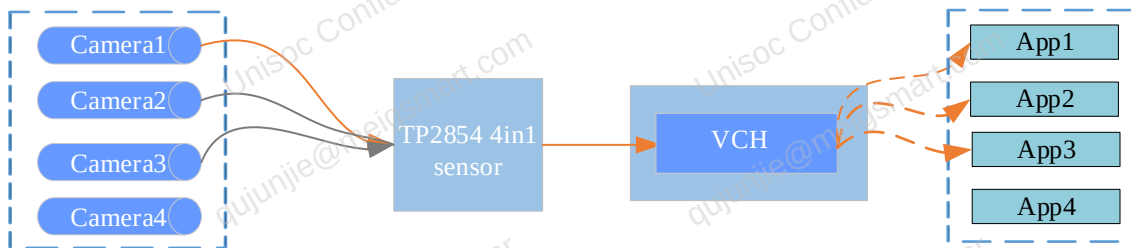
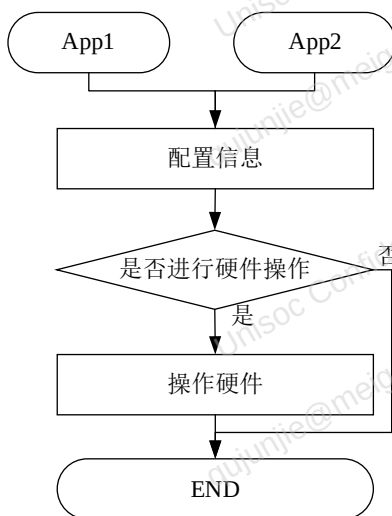


图2-15 多虚拟 Camera 流程图



2.4.3 配置文件

多虚拟 Camera 配置文件为 odm/etc/virtual_cam_cfg.json，适用于配置伪单帧和多虚拟通道模式。

2.4.3.1 文件介绍

以 virtual camera v2.3 为例，virtual_cam_cfg.json 文件中参数详见表 2-1，配置示例参见图 2-16、图 2-17。

表2-1 virtual_cam_cfg.json 参数含义

参数名	含义
switch	是否开启 MVC（Multi Virtual Camera，多虚拟 Camera）功能： • false：关闭 • true：开启
config_adapt	是否根据 App 需要的出图大小自动适配虚拟通道出图尺寸： • false：不自动适配 • true：自动适配

参数名			含义	
MVC_cfg	HW_cfg	HW_id	MVC 实际打开的硬件物理 ID，对应转接芯片 ID。	
		dump_en	是否转储 MVC 收到的来自硬件的图像： • false：不转储 • true：转储	
		one_to_x	解串芯片上摄像头的数量。	
		is_sns_stream	配置多虚拟通道模式和伪单帧模式： • false：伪单帧模式 • true：多虚拟通道模式	
	virtual_id_cfg	id	虚拟 id，上层应用通过该 id 来调用指定摄像头。	
		location	帧重组竖条纹位置，针对伪单帧模式。	
		sns_stream_order	虚拟通道编号，针对多虚拟通道模式。	
		sns_stream_w / sns_stream_h	虚拟通道 sensor 出图的宽、高	
		skip_num	前几帧中需要丢弃的帧数，针对多虚拟通道模式。	
		deinterlance：配置需要解交织算法的场景，针对多虚拟通道模式。		
		deinterlance	enable	• true：开启解交织 • false：关闭解交织
			vblank_y	奇场的高。
			vblank_delta_y	消隐行的高，如果没有，设置为 0。
		crop：配置裁剪信息，针对多虚拟通道模式。		
		crop	enable	• true：开启裁剪 • false：关闭裁剪
			start_x / start_y	开始裁剪的 x / y 坐标。
			width / height	裁剪的宽 / 高。
		dump_en	是否在 MVC 中转储当前 ID 的图像： • true：转储 • false：不转储	

图2-16 多虚拟通道配置示例

```

"version": "virtual camera v2.3",
"switch": false,
"config_adapt": false,
"MVC_cfg": [
  {
    "HW_cfg": {
      "HW_id": 1,
      "dump_en": false,
      "one_to_x": 4,
      "is_sns_stream": true
    },
    "virtual_id_cfg": [
      {
        "id": 2,
        "location": 0,
        "sns_stream_order": 0,
        "sns_stream_w": 1280,
        "sns_stream_h": 720,
        "skip_num": 0,
        "deinterlace": {
          "enable": false,
          "vblank_y": 350,
          "vblank_delta_y": 23
        },
        "crop": {
          "enable": false,
          "start_x": 0,
          "start_y": 0,
          "width": 1280,
          "height": 720
        },
        "dump_en": false
      }
    ]
  },

```

图2-17 伪单帧模式配置示例

```

{
  "version": "virtual camera v2.3",
  "switch": true,
  "config_adapt": true,
  "MVC_cfg": [
    {
      "HW_cfg": {
        "HW_id": 0,
        "dump_en": false,
        "one_to_x": 4,
        "is_sns_stream": false
      },
      "virtual_id_cfg": [
        {
          "id": 4,
          "location": 0,
          "sns_stream_order": 0,
          "sns_stream_w": 1280,
          "sns_stream_h": 720,
          "skip_num": 0,
          "deinterlace": {
            "enable": false,
            "vblank_y": 350,
            "vblank_delta_y": 23
          },
          "crop": {
            "enable": false,
            "start_x": 0,
            "start_y": 0,
            "width": 1280,
            "height": 720
          },
          "dump_en": false
        }
      ]
    }
  ],

```

2.4.3.2 配置方法

配置文件修改方法如下：

步骤 1 从设备中/`odm/etc/virtual_cam_cfg.json` 导出（pull）`virtual_cam_cfg.json` 文件。

步骤 2 配置相关参数。

步骤 3 将修改后的文件导入（push）到设备中。

步骤 4 重启设备或后台先结束 Camera Provider 服务再重启服务。

为了提高效率，MVC 会在 Camera Provider 服务启动时一次性解析配置文件，后续不再解析。修改配置文件后，需要重启 Camera Provider 服务才能生效。

----结束

3 测试方法

系统已经集成 cam2test 应用，用于测试车载 360AVM 环视功能。

图 3-1 展示了接一路摄像头，数据格式为垂直排列的 cam2test 预览界面。

图3-1 cam2test 应用测试界面



测试流程如下：

1. 正确 bring up 转接芯片，可通过 blue pattern 测试验证。
2. 接入相应数量的摄像头，可选竖直方式或田字格方式。
3. 使用 cam2test 测试预览录像拍照。
4. 修改 virtual_cam_cfg.json 文件并导入设备。
5. 安装 4 个 cam2test apk，打开对应虚拟 Camera 并测试拍照预览录像。

A

Sensor 介绍

A.1 功能开关

当前平台都支持 YUV sensor，通过宏开关控制。

路径：device/sprd/平台名/工程名/module/camera/md.mk

图3-2 YUV sensor 开关

```
1223 else ifeq ($(strip $(TARGET_BOARD_PLATFORM)),sp9863a)
1224 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_SHARKL3)
1225 else ifeq ($(filter $(strip $(TARGET_BOARD_PLATFORM)),ums510 ums312),)
1226 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_SHARKL5)
1227 else ifeq ($(filter $(strip $(TARGET_BOARD_PLATFORM)),ums518 ums512 ums7520 ums518-zebu),)
1228 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_SHARKL5P)
1229 else ifeq ($(strip $(TARGET_BOARD_PLATFORM)),roc1)
1230 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_ROC1)
1231 else ifeq ($(strip $(TARGET_BOARD_PLATFORM)),ums9230)
1232 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_SHARKL6)
1233 else ifeq ($(filter $(strip $(TARGET_BOARD_PLATFORM)),ums9520 ums9620),)
1234 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_SHARKL6P)
1235 else
1236 LOCAL_CFLAGS += -DPLATFORM_ID=$(LOCAL_PLATFORM_ID_GENERIC)
1237 endif
1238
1239 #check whether set PRIVATE_17
1240 ifeq ($(strip $(TARGET_BOARD_CAMERA_GALLIUM_USAGE_PRIVATE_17)),true)
1241 LOCAL_CFLAGS += -DCONFIG_CAMERA_USAGE_PRIVATE_17
1242 endif # CAMERA_GALLIUM_USAGE_PRIVATE_17
1243
1244 ifeq ($(strip $(TARGET_BOARD_YUV_SENSOR_SUPPORT)),true)
1245 LOCAL_CFLAGS += -DCONFIG_YUV_SUPPORT
1246 endif
1247
1248 ifeq ($(strip $(TARGET_BOARD_SPRD_MFNR_V4)), true)
1249 LOCAL_CFLAGS += -DCONFIG_CAMERA_MFNR_V4
1250 endif #TARGET_BOARD_SPRD_MFNR_V4
1251
1252 ifeq ($(strip $(TARGET_BOARD_SPRD_GO_32B_SUPPORT)),true)
1253 LOCAL_CFLAGS += -DCONFIG_GO_32B_AND_GO_32B_2C
1254 endif
1255
1256 #unisoc add for 1428744 Config 430 and CVBS camera by property
1257 ifeq ($(strip $(CONFIG_IVI_FEATURE)),true)
1258 LOCAL_CFLAGS += -DCONFIG_IVI_FEATURE
1259 LOCAL_CFLAGS += -DCONFIG_YUV_SUPPORT
1260 LOCAL_CFLAGS += -DCONFIG_VC_FEATURE
1261 LOCAL_CFLAGS += -DCONFIG_CVBS_DEINTERLACE
1262 endif
```

A.2 点亮配置

在 sensor_config.xml 中配置，无需配置效果参数节点。以 TP2854 为例，YUV sensor 点亮示例如下。

```
<CameraModuleCfg>
  <SlotId>0</SlotId>
  <SensorName>tp2854</SensorName>
  <Facing>BACK</Facing>
  <Orientation>90</Orientation>
  <Resource_cost>50</Resource_cost>
</CameraModuleCfg>
```

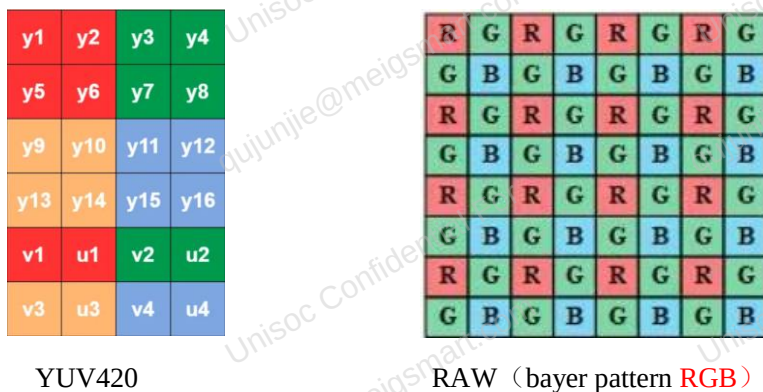
A.3 YUV sensor 和 RAW sensor 差异

A.3.1 功能差异

车载转接芯片可当作 YUV sensor。YUV sensor 输出 YUV 格式的图，RAW sensor 输出 RAW 格式的图。

YUV sensor 不经过平台 DCAM ISP 效果处理，RAW sensor 需要经过 DCAM ISP 效果处理。

图3-3 YUV 和 RAW 格式



平台接收 YUV422 格式或 RAW 格式的 sensor 数据，在 DCAM 或 ISP 转成 YUV420(NV21)。

A.3.2 点亮配置差异

在 sensor_config.xml 中配置，YUV 无需配置效果参数节点，除此之外 YUV sensor 和 RAW sensor 点亮的方式相同。

A.3.3 处理流程差异

YUV sensor 和 RAW sensor 处理流程的区别主要为效果，YUV 不进行效果处理，无需生成效果参数并且配置到代码中。

不同平台处理流程可能有所差异，具体以实际为准。

图3-4 YUV 和 RAW 处理流程差异

